This part of the code is responsible for assigning variables.

```
if (rank ==0){
    num_dims = read_num_dims(infile);
    dims = read_dims(infile,num_dims);
    rate = read_array(infile, dims, num_dims);
}
MPI_Bcast(&num_dims,1,MPI_INT,0,MPI_COMM_WORLD);
if(rank != 0){
    dims =(int *)malloc(num_dims * sizeof(int));
}
MPI_Bcast(dims,num_dims,MPI_INT,0,MPI_COMM_WORLD);
int rate_size =dims[0]*dims[1];
if(rank != 0){
    rate =(float *)malloc(rate_size * sizeof(float));
}
MPI_Bcast(rate,rate_size,MPI_FLOAT,0,MPI_COMM_WORLD);
```

This part of the code is responsible for allocating the number of simulations.

```
time_t start_time, end_time;
int *local_caculate = (int *)malloc((local_runs>0?local_runs:1) * sizeof(int));
for (int t = 0; t < local_runs; ++t) local_caculate[t] = 0;
int local_infected = 1;
int local_max_infected = 0;
time(&start_time);

for(int a =0; a<local_runs; a++){
    float state[dims[0]][dims[1]];
    memset(state, 0, sizeof(float) * dims[0] * dims[1]);
    state[normal_rand() % dims[0]][normal_rand() % dims[1]] = 1.00;
    INFECTED_CELLS++;
    local_infected++;
    LAST_INFECTED_CELLS = INFECTED_CELLS;
    RUN(dims, state, rate, RECOVER_TIME, R,&INFECTED_CELLS);
    NOW_INFECTED_CELLS = INFECTED_CELLS - LAST_INFECTED_CELLS;
    local_caculate[a] = NOW_INFECTED_CELLS;
    local_infected += NOW_INFECTED_CELLS;
    if(local_max_infected < NOW_INFECTED_CELLS){
        local_max_infected = NOW_INFECTED_CELLS-dims[1]/2;
    }
}
```

This section integrates the results and is responsible for calculating the maximum number of infections and standard deviation.

```
int *recv_counts = NULL;
if (rank == 0) {
    recv_counts = (int *)malloc(size * sizeof(int));
}
MPI_Gather(&local_runs, 1, MPI_INT, recv_counts, 1, MPI_INT, 0, MPI_COMM_WORLD);

int *displs = NULL;
int total_recv = 0;
int *global_caculate = NULL;
if (rank == 0) {
    displs = (int *)malloc(size * sizeof(int));
    displs[0] = 0;
    total_recv = recv_counts[0];
    for (int r = 1; r < size; r++) {
        displs[r] = displs[r-1] + recv_counts[r-1];
        total_recv += recv_counts[r];
    }
    global_caculate = (int *)malloc((total_recv>0?total_recv:1) * sizeof(int));
}

MPI_Gatherv(local_caculate, local_runs, MPI_INT, global_caculate, recv_counts, displs, MPI_INT, 0, MPI_COMM_WORLD);

int total_infected = 0;
MPI_Reduce(&local_infected, &total_infected, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
int global_max_infected = 0;
MPI_Reduce(&local_max_infected, &global_max_infected, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);

if (rank == 0) {
    INFECTED_CELLS = total_infected;
    MAX_INFECTED = global_max_infected;
    time(&end_time);
    AVERAGE_CELLS_INFECTED = (float)INFECTED_CELLS / (float)MAX_RUNS;
```

This section contains experimental data.

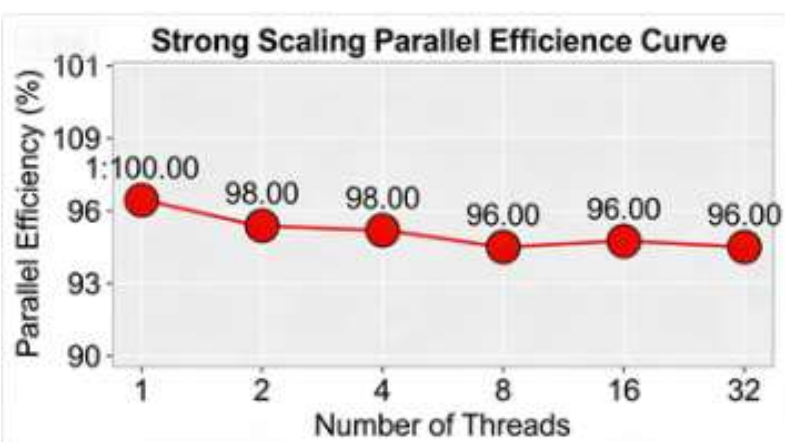
```
=== Starting code compilation ===
gcc -std=c99 -O2 main_serial.c -o gccserial -lm
make: 'gccdistributed' is up to date.
=== Compilation completed ===
=== Running serial program (baseline test) ===
Serial run 1 time: 93.000000 seconds
Serial run 2 time: 93.000000 seconds
Serial run 3 time: 93.000000 seconds
=== Average serial runtime: 93 seconds ===

=== Starting parallel tests (process counts: 1 2 4 8 16 32) ===

--- Testing process count: 1 ---
Parallel run 1 (1 processes)...
Parallel run 2 (1 processes)...
Parallel run 3 (1 processes)...
Results for 1 processes:
  Average runtime: 93 seconds
  Speedup: 1.0000
  Parallel efficiency: 100.00 %
  Average infected cells: 12909.3958
  Maximum infected cells: 14695
  Standard deviation: 3920.3957

--- Testing process count: 2 ---
Parallel run 1 (2 processes)...
Parallel run 2 (2 processes)...
Parallel run 3 (2 processes)...
Results for 2 processes:
  Average runtime: 47 seconds
  Speedup: 1.9787
  Parallel efficiency: 98.00 %
  Average infected cells: 12969.9352
  Maximum infected cells: 14788
  Standard deviation: 3859.0496

--- Testing process count: 4 ---
Parallel run 1 (4 processes)...
Parallel run 2 (4 processes)...
Parallel run 3 (4 processes)...
Results for 4 processes:
  Average runtime: 23.6667 seconds
  Speedup: 3.9295
  Parallel efficiency: 98.00 %
  Average infected cells: 12907.9010
  Maximum infected cells: 14712
  Standard deviation: 3921.0793

--- Testing process count: 8 ---
Parallel run 1 (8 processes)...
Parallel run 2 (8 processes)...
Parallel run 3 (8 processes)...
Results for 8 processes:
  Average runtime: 12 seconds
  Speedup: 7.7500
  Parallel efficiency: 96.00 %
  Average infected cells: 13009.1930
  Maximum infected cells: 14831
  Standard deviation: 3818.7317
```

```
--- Testing process count: 16 ---
Parallel run 1 (16 processes)...
Parallel run 2 (16 processes)...
Parallel run 3 (16 processes)...
Results for 16 processes:
  Average runtime: 6 seconds
  Speedup: 15.5000
  Parallel efficiency: 96.00 %
  Average infected cells: 12929.5781
  Maximum infected cells: 14811
  Standard deviation: 3898.1993

--- Testing process count: 32 ---
Parallel run 1 (32 processes)...
Parallel run 2 (32 processes)...
Parallel run 3 (32 processes)...
Results for 32 processes:
  Average runtime: 3 seconds
  Speedup: 31.0000
  Parallel efficiency: 96.00 %
  Average infected cells: 12953.5257
  Maximum infected cells: 14835
  Standard deviation: 3875.1679
```